

QA / Test Engineer 포트폴리오

Medusa 헤드리스 커머스 기능 검증

김정석

화면에 무엇이 뒀는지가 아니라, 그 아래 시스템 상태와 데이터 흐름이 기댓값과 일치하는지를 검증합니다. 그리고 무엇을 자동화하고 무엇을 자동화하지 않을지까지 판단하는 것을 QA의 일로 봅니다.

● **CI Passing** · GitHub Actions 정적 검증 통과

Repository: github.com/bbang2u/medusa_qa_portfolio

3줄 요약	
무엇을	Medusa 헤드리스 커머스의 구매 여정 전반(탐색·장바구니·인증·세션)을 기능 검증
무엇을 찾았나	결함 5건 — 비밀번호 정책 부재(Major)·품질 오폭시(Major) 등 유형이 다른 결함
자동화·CI	Playwright 자동화 16개 / GitHub Actions 정적 검증 초록 

목차

1. 프로젝트 개요	3
2. 나의 테스트 관점	4
3. 검증 프로세스	5
4. 핵심 결함 요약	6
5. 자동화 판단	7
6. 협업·품질 확장 관점	9
7. 한계와 성장 방향	10
8. 대표 테스트 케이스	11
9. 전체 케이스 요약 (16건)	13
부록 A. 결함 리포트 (5건)	14
부록 B. 테스트 계획 요약	19

1. 프로젝트 개요

대상: Medusa 2.15.5 — 셀프호스팅 헤드리스 커머스(스토어프론트 + Admin).

검증 범위: 상품 탐색 → 장바구니 → 인증/세션에 이르는 사용자 구매 여정 전반을, 화면 동작이 아니라 데이터 정합성·상태 전이·보안 관점에서 검증했다. (금액 합산, 세션 생명주기, 계정 열거 방지, 입력 검증의 계층 비대칭 등)

선정 이유: 금액·재고·세션처럼 틀리면 매출과 신뢰에 직접 타격이 가는 도메인이라, 표면 너머의 정합성을 검증하는 관점을 보여주기 위해 적합하다.

수행 범위: 테스트 계획부터 케이스 설계(16건)·수동 실행·결함 리포트(5건)·자동화 테스트(16개) 선별 구현까지 전 과정을 단독 수행. 설계자·실행자·자동화 엔지니어 역할을 분리해 각 단계 산출물을 남겼다.

2. 나의 테스트 관점

테스트는 "버그 찾기"가 아니라 "리스크를 줄이는 판단"이라고 본다. 모든 것을 동등하게 검증할 수는 없으므로, 어디가 틀리면 가장 크게 깨지는지를 먼저 정하고 거기에 검증을 집중했다.

① **리스크 기반 우선순위** — 금액·재고·인증처럼 틀리면 회복 불가능하거나 매출·신뢰에 직접 타격인 지점을 High로 두고 먼저 검증했다. 장바구니 합계 재계산·세션 생명주기·비밀번호 정책은 High, 표기·정렬 수준은 Med 이하.

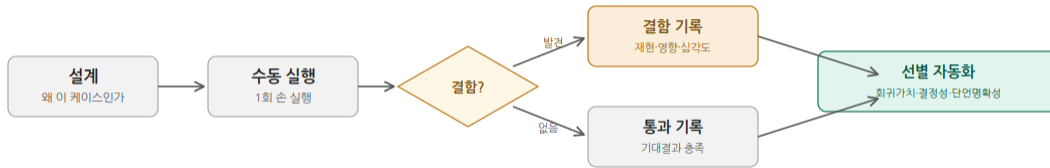
② **표면이 아니라 상태를 검증** — "화면에 뺀가"가 아니라 "그 아래 시스템 상태·데이터가 기댓값과 일치하는가"를 단언 기준으로 삼았다. 옵션 미선택을 품절로 표시하는 것(DEF-04), 프론트 통과 값이 Admin까지 영속되는 것(ADMIN-01)처럼 화면만 봐서는 안 보이는 결함을 잡았다.

③ **종료 기준 (Go/No-Go)** — 미해결 Blocker 0건 + 설계 케이스의 기대 결과 충족을 합격선으로 사전 정의하고, 그 기준으로 현재 상태를 판정한다.

④ **자동화는 개수가 아니라 선별** — 회귀가치·결과 결정성·단언 명확성 세 기준으로 자동화 대상을 골랐고, 기준에 못 미치는 것은 근거를 들어 제외했다.(상세: 5장)

3. 검증 프로세스

자동화를 먼저 짜지 않고, 설계 → 수동 실행 → 결함 기록 → 선별 자동화 순서를 엄격히 지켰다. 검증 설계의 사고 과정과 결함 발견 경위를 산출물로 남기기 위해서다.



자동화하지 않기로 한 판단도 근거와 함께 남긴다.

산출물 지도

산출물	내용	규모
테스트 계획서	범위·환경·심각도 기준·합격 기준	1부
케이스 명세서	11필드(설계 의도 포함) 케이스	16건
결함 리포트	재현·영향·심각도	5건
자동화 스크립트	Playwright, spec 7파일	테스트 16개

4. 핵심 결함 요약

발견한 결함 5건. 각 결함은 현상이 아니라 서비스·데이터에 미치는 영향을 기준으로 서술하고 심각도를 매겼다. 상세 재현 절차는 부록 A 참조.

ID	결함	영향	심각도
DEF-01	전화번호 필드 입력 검증 부재로 비정상 데이터 영속화	비정상 연락처가 고객 DB에 영속 저장 → 배송 ·CS 지장, Admin UI 깨짐	Minor
DEF-02	장바구니 수량 드롭다운에 동일 옵션값(value="1") 중복 노출	옵션 생성 로직의 경계 처리(off-by-one) 의심 신호. 기능 영향은 없음	Minor
DEF-03	회원가입 비밀번호 최소 길이 정책 부재	1글자 비밀번호로 계정 생성·로그인 가능 → brute-force 취약, 계정 탈취 위험	Major
DEF-04	상품 옵션 미선택 시 "Out of stock"으로 오표시	판매 가능 상품을 품절로 오인 → 구매 이탈, 매출 직결	Major
DEF-05	중복 이메일 가입 응답으로 계정 존재 여부 노출	등록된 이메일을 외부에서 식별 가능(계정 열거) → 표적 피싱 악용 여지. 위험도는 낮음	Minor

결함의 결을 다양하게 보았다: 있어야 할 것이 없음(DEF-03 정책 부재), 없는 것을 있다고 표시(DEF-04 오표시), 막긴 하되 정보를 흘림(DEF-05 계정 열거) — 단순 기능 버그가 아니라 유형이 다른 결함을 잡았다. 특히 DEF-05는 보안 리스크를 과장하지 않고 가입 폼의 트레이드오프를 함께 기록해, 맥락에 따라 심각도를 조정하는 판단을 보였다.

5. 자동화 판단

자동화는 개수가 목표가 아니다. 16개 케이스를 모두 수동 실행한 뒤, 세 기준으로 자동화 여부를 갈랐다.

선별 기준

- 회귀가치 — 핵심 경로(금액·인증·세션)에 있어 반복적으로 지켜야 하는가
- 결과 결정성 — 외부 의존·랜덤 없이 동일 입력에 동일 결과인가
- 단언 명확성 — 기댓값과의 일치를 코드로 정밀하게 표현할 수 있는가

단언 설계 — 코드로 보는 두 가지 원칙

"가격이 화면에 땀는가"가 아니라, 읽어온 가격 배열이 정렬된 배열과 동일한지를 단언한다.

기댓값과의 일치를 단언한다 — 가격 정렬 검증 (browse-sort.spec.ts)

```
test('가격 오름차순 정렬이 정확히 적용된다', async ({ page }) => {
  await page.goto(`${STORE_URL}?sortBy=price_asc`);
  const prices = await getPrices(page);
  expect(prices.length).toBeGreaterThanOrEqual(2);

  // 핵심 단언: 읽어온 배열이 같은 배열을 오름차순 정렬한 것과 동일한가
  const expectedAsc = [...prices].sort((a, b) => a - b);
  expect(prices).toEqual(expectedAsc);
});
```

waitForTimeout 같은 고정 대기 대신, 소계가 20→10으로 바뀔 때까지 상태 델타를 기다린다(교훈 #10).

시간이 아니라 상태 변화를 기다린다 — 삭제 후 재계산 (cart.spec.ts)

```
// 첫 번째 라인을 삭제 (삭제 버튼은 testid가 없어 행 내 유일한 button으로 잡음 - 교훈 #14)
await rows.first().locator('button').click();

// ★ 시간이 아니라 "변화"를 기다린다: Subtotal이 20 → 10으로
//   재계산될 때까지 자동 재시도 대기
await expect(
  page.locator('[data-testid="cart-subtotal"]')
).toHaveAttribute('data-value', '10');

await expect(rows).toHaveCount(1); // 라인 2 → 1 (경계값)
```

자동화하지 않은 것 (근거와 함께)

케이스	제외 근거
TC-CART-03 (빈 카트 차단)	라우팅 구조상 결정되어 회귀가치 낮음. 현재 404 차단이 개선 여지가 있어, 자동화로 고정하면 향후 UX 개선을 방해
TC-SESSION-03 (보호 페이지 직접 접근)	CART-03과 동일한 차단 패턴이라 회귀가치 중복. 인증 가드는 SESSION-01-02가 이미 커버
TC-ADMIN-01 (검증 비대칭)	Admin 인증 + 선행 가입에 의존해 독립 실행 어려움. 구조적 통찰을 보이는 케이스라 수동+증거가 적합
TC-CHECKOUT-01 (전화번호 검증)	의미 있는 검증(DB 영속 확인)에 Admin 경로 필요. 회귀 빈도 낮아 비용 대비 효용 낮음
DEF-02 (드롭다운 중복)	기능 영향 없는 UI 무결성 결함이라 회귀가치 미달. 수동 확인으로 충분

결함을 회귀 가드로 박제

일부 자동화는 의도적으로 FAIL하도록 작성했다. DEF-04(품질 오폭시), DEF-03(비번 정책 부재)은 올바른 동작을 단언하므로 현재는 실패하며, 이 실패가 곧 결함이 살아있다는 증거다. 결함이 수정되면 자동으로 통과로 바뀌어 회귀를 감지한다.

6. 협업·품질 확장 관점

이 프로젝트는 1인 검증이지만, 결함을 "전달 가능한 형태"로 남기는 것을 협업의 출발점으로 보았다.

개발자가 즉시 재현·수정할 수 있는 결함 리포트 — 모든 결함을 재현 절차·기대 vs 실제·영향·심각도 형식으로 작성했다. "화면이 깨진다"가 아니라 "어떤 조건에서, 무엇이, 왜 문제인가"를 담아 개발자가 바로 이슈로 받을 수 있게 했다.

단순 보고가 아닌 개선 제안 — 결함이 아닌 것도 더 나은 동작을 제안했다. 빈 카트·보호 페이지 직접 접근 시 404 대신 안내 페이지로 리다이렉트하는 편이 UX상 낫다는 제안, DEF-05의 트레이드오프 기록 등 맥락에 맞는 우선순위를 제안했다.

검증을 데이터 흐름 전체로 확장 — 화면 단위가 아니라, 프론트에서 통과한 입력이 Admin·DB까지 어떻게 영속되는지(TC-ADMIN-01 검증 비대칭)를 추적했다. 단일 화면 너머 시스템 경계를 보는 관점이다.

7. 한계와 성장 방향

이 포트폴리오의 한계를 명확히 인지하고 있으며, 한계를 아는 것 또한 검증자의 역량이라고 본다.

인지한 한계

- CI 환경 제약 — GitHub Actions 가 로컬 Medusa 인스턴스에 접속할 수 없어, 풀 E2E 회귀를 CI 에서 자동 실행하지는 못한다. 대신 워크플로 문법·구조 검증(의존성 설치·타입 체크·테스트 수집)까지 구성했다.
- 테스트 데이터 제약 — 시드 환경의 전 상품 재고가 충분해 실제 품질 상태의 검증은 환경상 재현하지 못했다.
- 검증 범위 — 결제 PG 연동·성능·크로스 브라우저는 의도적으로 범위에서 제외했다. 기능 정합성에 집중하기 위한 선택이며, 확장 시 우선 보강 영역이다.

강화하고 싶은 방향

- 단위·통합 레벨의 화이트박스 검증과 E2E 를 잇는 테스트 피라미드 전반의 설계 역량
- 테스트 설계 기법의 체계화 (ISTQB CTAL 학습 중)
- CI/CD 파이프라인 위에서 회귀 스위트가 실제로 도는 환경 구축

지향점 — 결함을 많이 찾는 테스터를 넘어, 어디를 검증하지 않아도 되는지까지 판단해 품질 비용을 최적화하는 QA 엔지니어를 지향한다.

8. 대표 테스트 케이스

16개 케이스 중, 결함을 발견했고 설계 의도가 뚜렷한 2건을 11필드 명세 전문으로 심는다. 나머지 14건은 9장 요약표 참조.

TC-DETAIL-01 / 다중 옵션 상품의 미선택 상태 표시 검증

전제조건	백엔드/스토어프론트 기동, 다중 옵션 상품(t-shirt: Color×Size, 실제 재고 있음) 존재
절차	1. /dk/products/t-shirt 진입 (옵션 미선택 초기 상태) 2. 구매 버튼 라벨·가격 확인 3. Color만 선택 후 버튼 확인 4. Size만 선택 후 버튼 확인 5. Color+Size 모두 선택 후 버튼 확인
테스트 데이터	t-shirt (Color: Black/White, Size: L/M/S/XL, 재고 있음)
기대 결과	옵션 미완성 상태(미선택, Color만, Size만)에서는 구매 버튼이 "Select variant" 등 미선택 안내를 표시해야 한다. 실제 재고가 있으므로 "Out of stock"은 표시되면 안 된다. Color+Size 모두 선택 시에만 "Add to cart" 활성화·가격 €10.00 표시.
설계 의도	다중 옵션 상품은 모든 옵션이 정해져야 variant가 특정되고 재고를 알 수 있다. 옵션 미완성 상태를 "품절"로 표시하면 미선택과 실제 품절이 구분 불가능해지고, 판매 가능 상품을 사용자가 품절로 오인해 이탈한다. 버튼 라벨이 "화면에 뜨나"가 아니라 시스템 상태(미선택 vs 품절)를 정확히 반영하는가를 본다.
기법	상태 전이(옵션 선택 조합별 버튼 상태) + 경계값(미완성 조합)
우선순위	High (판매 가능 상품의 품절 오인 → 매출 직결)
자동화	O. 옵션 조합별 버튼 상태가 결정적이고 단언이 명확하며 옵션 UI 회귀로 지킬 가치가 있다. 단, 현재는 결함 상태를 박제하는 의도적 실패 단언으로 작성(DEF-03 자동화 패턴과 동일).
실행 결과	Fail (결함 확인 → DEF-04). 옵션 미선택·Color만·Size만 상태에서 모두 "Out of stock" 표시. 판매 가능 상품의 품절 오표시 확인.

TC-AUTH-02 / 회원가입 입력 검증 (비밀번호 정책·필수값·이메일 형식)

전제조건	백엔드/스토어프론트 기동, 회원가입 폼(/dk/account → Join us) 진입 가능, 매 실행 시 미가입 이메일 확보
절차	1. 회원가입 폼 진입 2. (baseline) 전 필드 정상값 → 가입 성공해야 3. 비밀번호 1글자 → 거부되어야 4. 필수 필드(이메일) 누락 → 거부되어야 5. 잘못된 이메일 형식(@ 없음) → 거부되어야
테스트 데이터	정상: email=test+{타임스탬프}@example.com, password=ValidPass123 / 짧은 비번: a / 형식 오류: notanemail
기대 결과	baseline은 가입 성공·계정 페이지 진입. 짧은 비번은 거부+길이 정책 안내. 이메일 누락은 거부(필수값). 형식 오류는 거부(형식).
설계 의도	회원이입은 모든 사용자 데이터의 입구다. 검증이 부실하면 약한 비밀번호 계정·잘못된 형식 데이터가 그대로 영속 저장된다. 특히 비밀번호 정책은 계정 보안의 1차 방어선이라 최소 길이 검증을 우선 확인한다. baseline을 함께 두어, 거부가 "전부 막는 버그"가 아니라 "비정상만 골라 막는지"를 대조 검증한다.
기법	동등 분할(정상/비정상 입력 클래스) + 경계값(비밀번호 길이) + 예외 처리(필수값 형식)
우선순위	High (비밀번호 정책은 보안 직결)
자동화	○ — 입력→결과 결정적, 단언 명확. 매 실행 유니크 이메일 필요(타임스탬프). register.spec.ts에 시나리오별 test() 분리.
실행 결과	Fail (결함 발견). baseline 정상 가입 ☒ / 짧은 비번 거부되지 않고 가입 성공 → DEF-03. 이메일 누락·형식 거부는 자동화로 확정.

9. 전체 케이스 요약 (16 건)

설계·수동 실행한 16개 케이스 전체. 우선순위와 자동화 여부는 별개 축으로 판단했다(자동화 = 회귀가치 + 결정성 + 단언 명확성).

TC-ID	제목	우선순위	자동화	실행 결과
TC-BROWSE-01	상점 가격 정렬 정확성	Med	O	Pass
TC-BROWSE-02	카테고리 상품 격리	Med	O	Pass
TC-CART-02	수량 변경 합계 재계산	High	O	Pass
TC-CART-03	빈 카트 체크아웃 차단	Med	X	Pass(관찰)
TC-CART-04	다중 상품 소계 합산	High	O	Pass
TC-CART-05	항목 삭제 후 재계산	High	O	Pass
TC-CHECKOUT-01	전화번호 입력 검증	Low~Med	X	Fail → DEF-01
TC-AUTH-01	로그인 에러 메시지 일관성	High	O	Pass
TC-AUTH-02	회원가입 입력 검증	High	O	Fail → DEF-03
TC-AUTH-03	중복 이메일 가입 차단	Med	O	부분 Fail → DEF-05
TC-ADMIN-01	검증 비대칭(Admin 영속)	Med	X	Fail → DEF-01
TC-SESSION-01	세션 페이지 이동 지속성	High	O	Pass
TC-SESSION-02	로그아웃 세션 종료	High	O	Pass
TC-SESSION-03	보호 페이지 직접 접근 차단	Med	X	Pass(관찰)
TC-DETAIL-01	옵션 미선택 품질 오폭시	High	O	Fail → DEF-04
TC-DETAIL-02	목록·상세 가격 일관성	Med	O	Pass

부록 A. 결함 리포트 (5 건)

결함 5건의 전체 리포트. 각 결함은 재현 절차·기대 vs 실제·영향을 포함해 개발자가 바로 재현·수정할 수 있도록 작성했다.

DEF-01 전화번호 필드 입력 검증 부재로 비정상 데이터 영속화

심각도 Minor · 우선순위 Low~Medium · 재현율 100%

요약	체크아웃 전화번호 필드가 길이·문자 타입 검증 없이 임의 입력을 허용하며, 해당 값으로 주문이 완료되고 고객 정보로 영속 저장됨. Admin 고객 상세에서 UI 레이아웃 오버플로우까지 유발.
재현 절차	1. 상품을 장바구니에 담고 체크아웃 진입 2. 이메일 등 필수 필드는 정상 입력 3. 전화번호 필드에 '2'를 100자 이상 반복 입력 (또는 문자·특수문자 입력) 4. 주문을 끝까지 진행 5. Admin → Customers → 해당 고객 상세 진입하여 Phone 필드 확인
기대 결과	전화번호 필드에 숫자 외 문자·특수문자는 거부되거나, 합리적 최대 길이(예: 국제번호 기준 15~20자)로 제한되어야 한다. 비정상 형식은 제출 시점에 검증 에러로 차단되어야 한다.
실제 결과	100자리 숫자·문자·특수문자가 모두 검증 없이 통과되어 주문이 정상 완료되고, Admin 고객 상세 Phone 필드에 그대로 저장됨. 저장된 값이 셀 경계를 넘어 우측으로 오버플로우되어 가독성 저하.
영향	데이터 무결성: 유효하지 않은 연락처가 고객 DB에 영속 저장됨. 운영: 배송·CS 단계에서 고객 연락 시 지장 가능. 표시: Admin UI 레이아웃 깨짐. (이메일 검증은 건고하므로 주문 확인 발송 자체는 가능 → Blocker는 아님)
출처	TC-CHECKOUT-01 / TC-ADMIN-01

DEF-02 장바구니 수량 드롭다운에 동일 옵션값(value="1") 중복 노출

심각도 Minor · 우선순위 Low · 재현율 100%

요약	장바구니 수량 선택 드롭다운 옵션이 1~10이어야 하나, 마지막에 value="1" 옵션이 한 번 더 붙어 총 11개(1,2,...,10,1)가 렌더링됨. 기능 동작은 정상이나 옵션 목록 무결성에 결함.
재현 절차	1. 상품을 장바구니에 담고 /dk/cart 진입 2. 수량 드롭다운(product-select-button)의 <option> 노드를 F12 Elements에서 확인 3. 렌더링된 option 개수와 각 value를 확인
기대 결과	수량 옵션은 1부터 상한(10)까지 중복 없이 정확히 10개여야 하며, 각 option의 value가 유일해야 한다.
실제 결과	option이 11개 렌더링됨(1~10 뒤에 value="1"이 한 번 더 추가). 동일 value("1")가 목록에 2회 존재. 단, 수량 선택·합계 재계산 등 기능 동작은 모두 정상.
영향	데이터 무결성: 중복 값 존재로 옵션 생성 로직의 경계 처리(off-by-one) 의심. 사용자: 의미 없는 "1" 항목 노출로 경미한 혼란. 기능: 실제 동작에는 영향 없음.
출처	TC-CART-02

DEF-03 회원가입 비밀번호 최소 길이 정책 부재

심각도 **Major** · 우선순위 High · 재현율 100%

요약	회원가입 시 비밀번호에 길이 제한이 없어, 1글자 비밀번호로도 계정이 정상 생성되고 로그인 까지 가능하다.
재현 절차	1. /dk/account에서 "Join us"로 회원가입 폼 진입 2. First/Last name, Email(미가입 주소)을 정상값으로 입력 3. Password에 'a'(1글자) 입력 후 Join 4. 가입 완료 후 동일 이메일 + 비밀번호 'a'로 로그인 시도
기대 결과	비밀번호가 최소 길이 기준(예: 8자) 미달이므로 가입이 거부되고, 길이 정책 안내 메시지가 표시되어야 한다.
실제 결과	1글자 비밀번호로 가입이 완료되고 해당 계정으로 로그인까지 성공한다. 최소 길이 검증·안내가 전혀 없다. (2글자에서도 동일 확인)
영향	비밀번호 강도 정책 부재로 계정이 무차별 대입(brute-force) 공격에 취약. 인증 보안의 기본 방어선이 없는 상태로, 실제 서비스라면 계정 탈취 위험으로 직결된다.
출처	TC-AUTH-02

DEF-04 상품 옵션 미선택 시 "Out of stock"으로 오표시 (미특정 variant와 품절 혼동)

심각도 **Major** · 우선순위 High · 재현율 100% (다중 옵션 상품 전체)

요약	다중 옵션 상품(t-shirt: Color×Size) 상세 진입 시, 옵션을 선택하지 않은 초기 상태에서 구매 버튼이 "Out of stock"으로 표시됨. Color만 또는 Size만 선택해도 동일. 모두 선택해야 "Add to cart"로 전환됨.
재현 절차	1. /dk/products/t-shirt 진입 (옵션 미선택 초기 상태) 2. 구매 버튼 라벨 확인 → "Out of stock" 3. Color(Black)만 선택 → 여전히 "Out of stock" 4. Size(L)만 선택 → 여전히 "Out of stock" 5. Color+Size 모두 선택 → "Add to cart" 활성화 (실제 재고 있음)
기대 결과	옵션 미완성 상태에서는 "Select variant"/"옵션을 선택하세요" 등 미선택 안내가 표시되어야 한다. 실제 재고가 있는 상품을 "Out of stock"으로 표시하면 안 된다.
실제 결과	미선택·Color만·Size만 상태에서 모두 "Out of stock" 표시. 판매 가능 상품의 품절 오표시 확인.
영향	판매 가능 상품을 품절로 오인하게 만들어 구매 이탈 유발(매출 직결). 진짜 품절 variant와 미선택 상태가 동일 메시지라 구분 불가. 가격은 "From €10.00"으로 정상 안내되면서 버튼만 품절이라 메시지 간 모순.
출처	TC-DETAIL-01

DEF-05 중복 이메일 가입 응답으로 계정 존재 여부 노출 (Account Enumeration)

심각도 Minor · 우선순위 Low · 재현율 100%

요약	이미 가입된 이메일로 회원가입을 시도하면 가입은 정상 차단되나, 화면에 "Identity with email already exists" 메시지가 노출되어 해당 이메일이 이미 등록되어 있다는 사실을 그대로 알려줌. 미가입 이메일 시도(정상 진행)와 응답이 구분되어, 외부에서 특정 이메일의 가입 여부를 식별할 수 있음.
재현 절차	1. /dk/account → "Join us"로 회원가입 폼 진입 2. 이미 가입된 이메일(bbang3u@naver.com) + 유효 비번 + 임의 이름 입력 3. Join 클릭 → 가입 차단됨(대시보드 미진입) 4. 화면 메시지 확인 → "Identity with email already exists" 노출 5. (대조) 미가입 이메일로 동일 시도 → 가입 진행, 다른 응답
기대 결과	중복 가입은 차단하되, 응답만으로 계정 존재 여부가 직접 드러나지 않는 것이 보안상 바람직하다. (단, 회원가입 폼은 중복을 사용자에게 알려야 하는 본질적 트레이드오프가 있어, login·비밀번호 재설정 플로우만큼 엄격히 차단하기는 어렵다.)
실제 결과	중복 가입은 정상 차단되나, 내부 에러 메시지가 그대로 노출되어 계정 존재 여부가 식별 가능.
영향	등록된 이메일을 외부에서 식별할 수 있어 표적 피싱·크리덴셜 스테핑의 사전 정보로 악용될 여지. 다만 가입 폼 특성상 흔한 동작이고 핵심 기능(중복 차단)은 정상이므로 실질 위험도는 낮음. 과장하지 않고 관찰 수준의 보안 리스크로 기록함.
출처	TC-AUTH-03

부록 B. 테스트 계획 요약

검증 활동의 기준 문서인 테스트 계획서(v1.0)의 핵심을 옮긴다. 전체 문서는 레포의 테스트 계획서를 참조.

대상 환경

항목	내용
대상 시스템	Medusa 2.15.5 (셀프호스팅 헤드리스 커머스)
아키텍처	Turborepo 모노레포 (backend + storefront 분리)
백엔드 / 스토어프론트	포트 9000 (Admin /app) / 포트 8000 (Next.js)
자동화 도구	Playwright (Chromium), GitHub Actions CI
언어/런타임	TypeScript 5.6, React 18, Node.js 22

검증 범위 (In Scope)

영역	주요 검증 내용	우선순위
구매 플로우	상품 상세 → 장바구니 → 수량 변경 → 체크아웃 입력 검증	최우선
상품 탐색	목록↔상세 데이터 정합성, 필터	높음
회원/인증	가입·로그인·게스트 체크아웃 분기	중간
Admin 기능	상품·재고·주문의 스토어 반영 정합성	중간
세션/영속성	새로고침·재방문 시 상태 유지	낮음

검증 제외 (Out of Scope) — 제외 사유 명시

제외 항목	사유
결제 PG 연동	외부 의존성으로 결과 비결정적, 검증 통제 불가
성능/부하	기능 검증과 목표 상이, 별도 도구·환경 필요
보안 취약점(침투)	별도 보안 검증 활동의 영역
크로스 브라우저	본 사이클은 Chromium 기준 기능 정합성에 집중

결함 심각도 기준

결함은 '결과의 회복 불가능성'과 '핵심 비즈니스 가치 차단' 정도로 분류한다. 특히 잘못된 금액·수량이 그대로 확정되는 경우를 Blocker로 본다.

심각도	정의	예시
Blocker	구매 진행 불가, 또는 금액·수량이 틀리게 확정됨	합계 오계산 주문 성립
Critical	핵심 기능은 되나 우회 필요, 표시 데이터 정확성 붕괴	목록↔상세 가격 불일치
Major	기능은 되나 예외 처리·검증 메시지 미흡	옵션 미선택 안내 부재
Minor	사용성·표기 수준의 경미한 결함	문구 오타, 정렬 어긋남

합격 / 불합격 기준 (Go/No-Go)

합격: Blocker 결함이 0건이며, 설계한 케이스의 기대 결과가 모두 충족됨. / 불합격: 미해결 Blocker 결함이 1건 이상 존재함.